

```

import os, shutil, cv2
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from glob import glob
import seaborn as sns
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.layers import Multiply, GlobalAvgPool2D, Reshape
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import confusion_matrix

# STEP 2: Split Dataset
#original_dataset_path = "COVID-19_Radiography_Dataset"
original_dataset_path = "Covid Datasets1"
base_dir = "lung_xray_split"

def split_data():
    if os.path.exists(base_dir):
        shutil.rmtree(base_dir)
    for folder in ["train", "val", "test"]:
        for category in ["Covid", "Normal", "Viral Pneumonia"]:
            os.makedirs(os.path.join(base_dir, folder, category),
exist_ok=True)

    for category in ["Covid", "Normal", "Viral Pneumonia"]:
        category_path = os.path.join(original_dataset_path, category)
        if not os.path.exists(category_path):
            print(f"Warning: {category_path} does not exist!
Skipping...")
            continue
        images = os.listdir(category_path)
        np.random.shuffle(images)
        train_split = int(0.8 * len(images))
        val_split = int(0.9 * len(images))
        for i, img in enumerate(images):
            src = os.path.join(category_path, img)
            if i < train_split:
                dest = os.path.join(base_dir, "train", category, img)
            elif i < val_split:
                dest = os.path.join(base_dir, "val", category, img)
            else:
                dest = os.path.join(base_dir, "test", category, img)
            shutil.copy(src, dest)

split_data()

```

```

# STEP 3: Data Augmentation
batch_size = 32
img_size = (224, 224)

train_datagen = ImageDataGenerator(
    rescale=1./255,
    horizontal_flip=True,
    rotation_range=20,
    zoom_range=0.2
)

val_test_datagen = ImageDataGenerator(rescale=1./255)

data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2),
])

# STEP 4: SE Block
def se_block(inputs, ratio=8):
    filters = inputs.shape[-1]
    se_shape = (1, 1, filters)
    se = GlobalAvgPool2D()(inputs)
    se = Reshape(se_shape)(se)
    se = layers.Dense(filters // ratio, activation='relu')(se)
    se = layers.Dense(filters, activation='sigmoid')(se)
    return Multiply()([inputs, se])

# STEP 5: CNN with L2 Regularizer
def create_model():
    inputs = layers.Input(shape=(224, 224, 3))
    x = data_augmentation(inputs)
    x = layers.Conv2D(32, 3, padding="same", activation='relu',
kernel_regularizer=regularizers.l2(0.01))(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(64, 3, padding="same", activation='relu',
kernel_regularizer=regularizers.l2(0.01))(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(128, 3, padding="same", activation='relu',
kernel_regularizer=regularizers.l2(0.01), name="last_conv")(x)
    x = layers.MaxPooling2D(2)(x)
    x = se_block(x)
    x = layers.Flatten()(x)
    x = layers.Dense(128, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(3, activation='softmax')(x)
    return Model(inputs, outputs)

```

```
C:\Users\UmarIqbal\anaconda3\lib\site-packages\scipy\__init__.py:155:
UserWarning: A NumPy version >=1.18.5 and <1.25.0 is required for this
version of SciPy (detected version 1.26.4
```

```
warnings.warn(f"A NumPy version >={np_minversion} and
<{np_maxversion}")
```

```
# STEP 6: KFold Training
```

```
def create_dataframe(data_dir):
    paths, labels = [], []
    for label in ["Covid", "Normal", "Viral Pneumonia"]:
        class_path = os.path.join(data_dir, label)
        if not os.path.exists(class_path):
            print(f"Warning: {class_path} does not exist!")
            continue
        # Match multiple image formats
        extensions = ('*.jpg', '*.jpeg', '*.png', '*.bmp')
        for ext in extensions:
            for img_path in glob(os.path.join(class_path, ext)):
                paths.append(img_path)
                labels.append(label)
    return pd.DataFrame({"filename": paths, "class": labels})
```

```
df = create_dataframe(os.path.join(base_dir, "train"))
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
early_stopping = EarlyStopping(
    monitor='val_accuracy',
    patience=5,
    restore_best_weights=True
)
```

```
fold = 1
for train_idx, val_idx in kf.split(df, df['class']):
    print(f"\n--- Fold {fold} ---")
    train_df = df.iloc[train_idx]
    val_df = df.iloc[val_idx]

    train_gen = train_datagen.flow_from_dataframe(
        train_df,
        x_col='filename',
        y_col='class',
        target_size=img_size,
        class_mode='categorical', # for one-hot encoding
        batch_size=batch_size,
        shuffle=True
    )

    val_gen = val_test_datagen.flow_from_dataframe(
        val_df,
```

```

x_col='filename',
y_col='class',
target_size=img_size,
class_mode='categorical',
batch_size=batch_size,
shuffle=False
)

model = create_model()
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

model.fit(train_gen, validation_data=val_gen, epochs=10,
callbacks=[early_stopping])
fold += 1

```

--- Fold 1 ---
Found 202 validated image filenames belonging to 3 classes.
Found 51 validated image filenames belonging to 3 classes.

C:\Users\UmarIqbal\anaconda3\lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.
self._warn_if_super_not_called()

Epoch 1/10
7/7 ————— 122s 12s/step - accuracy: 0.3967 - loss: 2.5949 - val_accuracy: 0.5098 - val_loss: 2.1615
Epoch 2/10
7/7 ————— 62s 9s/step - accuracy: 0.4317 - loss: 2.1103 - val_accuracy: 0.4314 - val_loss: 1.8627
Epoch 3/10
7/7 ————— 62s 8s/step - accuracy: 0.3865 - loss: 1.9299 - val_accuracy: 0.6667 - val_loss: 1.6979
Epoch 4/10
7/7 ————— 56s 8s/step - accuracy: 0.5523 - loss: 1.7248 - val_accuracy: 0.5882 - val_loss: 1.5560
Epoch 5/10
7/7 ————— 86s 7s/step - accuracy: 0.5819 - loss: 1.5825 - val_accuracy: 0.6667 - val_loss: 1.3810
Epoch 6/10
7/7 ————— 81s 7s/step - accuracy: 0.6125 - loss: 1.4149 - val_accuracy: 0.8627 - val_loss: 1.1409
Epoch 7/10
7/7 ————— 56s 7s/step - accuracy: 0.6235 - loss: 1.3537

```
- val_accuracy: 0.4902 - val_loss: 1.3882
Epoch 8/10
7/7 _____ 84s 8s/step - accuracy: 0.6724 - loss: 1.2122
- val_accuracy: 0.8039 - val_loss: 1.0467
Epoch 9/10
7/7 _____ 87s 8s/step - accuracy: 0.7238 - loss: 1.1559
- val_accuracy: 0.8627 - val_loss: 0.8884
Epoch 10/10
7/7 _____ 76s 8s/step - accuracy: 0.7486 - loss: 1.0516
- val_accuracy: 0.7843 - val_loss: 0.8921
```

--- Fold 2 ---

Found 202 validated image filenames belonging to 3 classes.

Found 51 validated image filenames belonging to 3 classes.

```
C:\Users\UmarIqbal\anaconda3\lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
  self._warn_if_super_not_called()
```

```
Epoch 1/10
7/7 _____ 67s 7s/step - accuracy: 0.3455 - loss: 2.7550
- val_accuracy: 0.2745 - val_loss: 2.1745
Epoch 2/10
7/7 _____ 75s 8s/step - accuracy: 0.4526 - loss: 2.1247
- val_accuracy: 0.4314 - val_loss: 1.9572
Epoch 3/10
7/7 _____ 52s 7s/step - accuracy: 0.4517 - loss: 1.9230
- val_accuracy: 0.4314 - val_loss: 1.7592
Epoch 4/10
7/7 _____ 48s 7s/step - accuracy: 0.4631 - loss: 1.7665
- val_accuracy: 0.7255 - val_loss: 1.5938
Epoch 5/10
7/7 _____ 49s 8s/step - accuracy: 0.5974 - loss: 1.5486
- val_accuracy: 0.6863 - val_loss: 1.3246
Epoch 6/10
7/7 _____ 53s 7s/step - accuracy: 0.5675 - loss: 1.4093
- val_accuracy: 0.7647 - val_loss: 1.1683
Epoch 7/10
7/7 _____ 85s 7s/step - accuracy: 0.6249 - loss: 1.3150
- val_accuracy: 0.6471 - val_loss: 1.1399
Epoch 8/10
7/7 _____ 85s 7s/step - accuracy: 0.5828 - loss: 1.2170
- val_accuracy: 0.7843 - val_loss: 1.0061
Epoch 9/10
7/7 _____ 78s 7s/step - accuracy: 0.5981 - loss: 1.2394
- val_accuracy: 0.8039 - val_loss: 0.9029
```

```
Epoch 10/10
7/7 _____ 51s 7s/step - accuracy: 0.6631 - loss: 1.1201
- val_accuracy: 0.8431 - val_loss: 0.8690
```

--- Fold 3 ---

Found 202 validated image filenames belonging to 3 classes.

Found 51 validated image filenames belonging to 3 classes.

```
C:\Users\UmarIqbal\anaconda3\lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
```

```
self._warn_if_super_not_called()
```

```
Epoch 1/10
7/7 _____ 75s 9s/step - accuracy: 0.3367 - loss: 2.4255
- val_accuracy: 0.4510 - val_loss: 2.1179
```

```
Epoch 2/10
7/7 _____ 60s 8s/step - accuracy: 0.4554 - loss: 2.0667
- val_accuracy: 0.5490 - val_loss: 1.8138
```

```
Epoch 3/10
7/7 _____ 51s 7s/step - accuracy: 0.5192 - loss: 1.8180
- val_accuracy: 0.6275 - val_loss: 1.5534
```

```
Epoch 4/10
7/7 _____ 89s 7s/step - accuracy: 0.5211 - loss: 1.6194
- val_accuracy: 0.7059 - val_loss: 1.3572
```

```
Epoch 5/10
7/7 _____ 82s 7s/step - accuracy: 0.5767 - loss: 1.4327
- val_accuracy: 0.7451 - val_loss: 1.0950
```

```
Epoch 6/10
7/7 _____ 85s 8s/step - accuracy: 0.6331 - loss: 1.2384
- val_accuracy: 0.5294 - val_loss: 1.3148
```

```
Epoch 7/10
7/7 _____ 77s 8s/step - accuracy: 0.6308 - loss: 1.2505
- val_accuracy: 0.7059 - val_loss: 0.9658
```

```
Epoch 8/10
7/7 _____ 85s 7s/step - accuracy: 0.6350 - loss: 1.0869
- val_accuracy: 0.7647 - val_loss: 0.8417
```

```
Epoch 9/10
7/7 _____ 57s 7s/step - accuracy: 0.6506 - loss: 1.0439
- val_accuracy: 0.7647 - val_loss: 0.8781
```

```
Epoch 10/10
7/7 _____ 82s 7s/step - accuracy: 0.6681 - loss: 1.0851
- val_accuracy: 0.7451 - val_loss: 0.7739
```

--- Fold 4 ---

Found 203 validated image filenames belonging to 3 classes.

Found 50 validated image filenames belonging to 3 classes.

```
C:\Users\UmarIqbal\anaconda3\lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
```

```
self._warn_if_super_not_called()
```

Epoch 1/10

```
7/7 _____ 69s 7s/step - accuracy: 0.3068 - loss: 2.8976
- val_accuracy: 0.4400 - val_loss: 2.1908
```

Epoch 2/10

```
7/7 _____ 62s 8s/step - accuracy: 0.4564 - loss: 2.1479
- val_accuracy: 0.4400 - val_loss: 1.9269
```

Epoch 3/10

```
7/7 _____ 78s 7s/step - accuracy: 0.4976 - loss: 1.9647
- val_accuracy: 0.6600 - val_loss: 1.7915
```

Epoch 4/10

```
7/7 _____ 83s 7s/step - accuracy: 0.6001 - loss: 1.7456
- val_accuracy: 0.6800 - val_loss: 1.5405
```

Epoch 5/10

```
7/7 _____ 75s 6s/step - accuracy: 0.6276 - loss: 1.5804
- val_accuracy: 0.7400 - val_loss: 1.3539
```

Epoch 6/10

```
7/7 _____ 54s 7s/step - accuracy: 0.6029 - loss: 1.4684
- val_accuracy: 0.6800 - val_loss: 1.2277
```

Epoch 7/10

```
7/7 _____ 50s 8s/step - accuracy: 0.5578 - loss: 1.5045
- val_accuracy: 0.6600 - val_loss: 1.3650
```

Epoch 8/10

```
7/7 _____ 55s 7s/step - accuracy: 0.6457 - loss: 1.3108
- val_accuracy: 0.7000 - val_loss: 1.1156
```

Epoch 9/10

```
7/7 _____ 48s 6s/step - accuracy: 0.6047 - loss: 1.3142
- val_accuracy: 0.7200 - val_loss: 1.0486
```

Epoch 10/10

```
7/7 _____ 54s 7s/step - accuracy: 0.6359 - loss: 1.1702
- val_accuracy: 0.8800 - val_loss: 0.9827
```

--- Fold 5 ---

Found 203 validated image filenames belonging to 3 classes.

Found 50 validated image filenames belonging to 3 classes.

```
C:\Users\UmarIqbal\anaconda3\lib\site-packages\keras\src\trainers\
data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor.
`**kwargs` can include `workers`, `use_multiprocessing`,
`max_queue_size`. Do not pass these arguments to `fit()`, as they will
be ignored.
```

```
self._warn_if_super_not_called()
```

```

Epoch 1/10
7/7 _____ 73s 8s/step - accuracy: 0.3437 - loss: 2.7032
- val_accuracy: 0.4200 - val_loss: 2.1282
Epoch 2/10
7/7 _____ 57s 7s/step - accuracy: 0.3189 - loss: 2.0982
- val_accuracy: 0.4200 - val_loss: 1.8741
Epoch 3/10
7/7 _____ 82s 7s/step - accuracy: 0.4444 - loss: 1.8771
- val_accuracy: 0.6400 - val_loss: 1.6330
Epoch 4/10
7/7 _____ 53s 7s/step - accuracy: 0.5872 - loss: 1.6167
- val_accuracy: 0.7600 - val_loss: 1.2841
Epoch 5/10
7/7 _____ 53s 7s/step - accuracy: 0.5705 - loss: 1.4614
- val_accuracy: 0.6800 - val_loss: 1.2243
Epoch 6/10
7/7 _____ 52s 7s/step - accuracy: 0.5797 - loss: 1.3671
- val_accuracy: 0.8000 - val_loss: 1.1319
Epoch 7/10
7/7 _____ 73s 10s/step - accuracy: 0.6389 - loss:
1.2523 - val_accuracy: 0.7600 - val_loss: 0.9450
Epoch 8/10
7/7 _____ 84s 10s/step - accuracy: 0.6509 - loss:
1.2017 - val_accuracy: 0.7600 - val_loss: 0.9085
Epoch 9/10
7/7 _____ 66s 9s/step - accuracy: 0.6806 - loss: 1.0959
- val_accuracy: 0.8200 - val_loss: 0.8223
Epoch 10/10
7/7 _____ 61s 8s/step - accuracy: 0.6719 - loss: 1.0068
- val_accuracy: 0.8600 - val_loss: 0.7505

```

STEP 7: Test Evaluation and Confusion Matrix

```

test_dataset = val_test_datagen.flow_from_directory(
    os.path.join(base_dir, "test"),
    target_size=img_size,
    batch_size=batch_size,
    class_mode="categorical",
    shuffle=False
)

test_loss, test_acc = model.evaluate(test_dataset)
print(f"Test Accuracy: {test_acc * 100:.2f}%")

y_true = test_dataset.classes
y_pred = np.argmax(model.predict(test_dataset), axis=-1)

cm = confusion_matrix(y_true, y_pred)
labels = list(test_dataset.class_indices.keys())
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=labels,
yticklabels=labels)

```



```
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()
```

Found 32 images belonging to 3 classes.

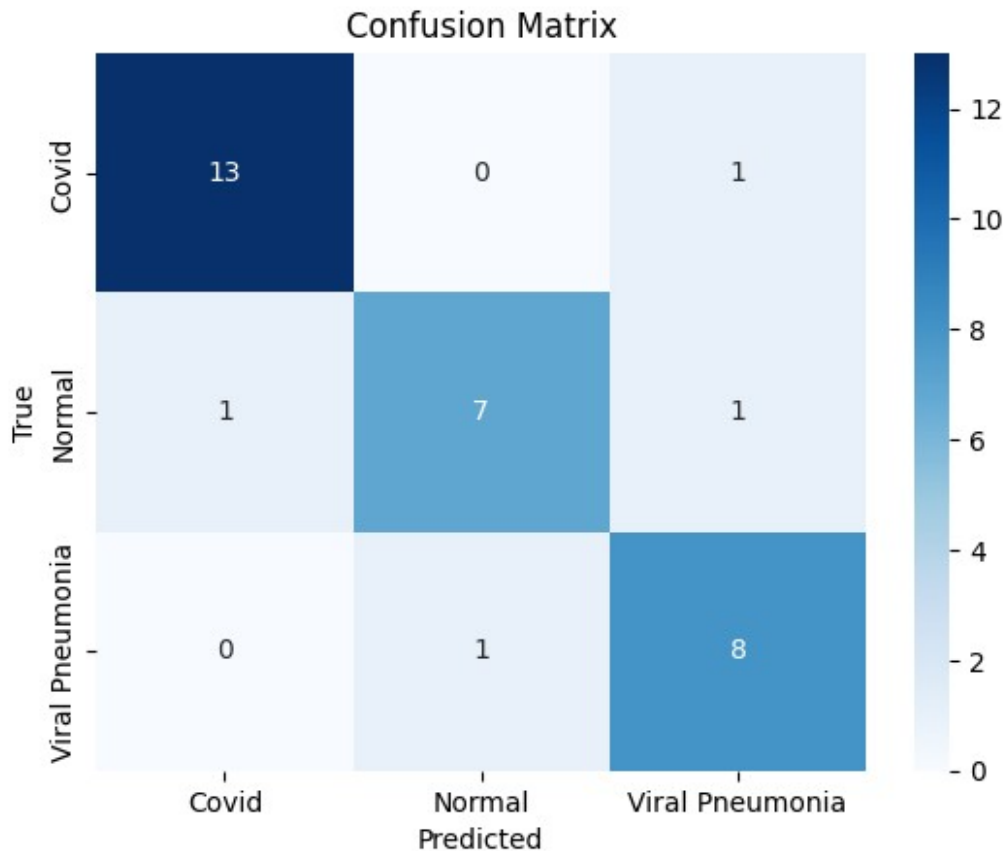
1/1

 6s 6s/step - accuracy: 0.8750 - loss: 0.8113

Test Accuracy: 87.50%

1/1

 7s 7s/step



STEP 8: Grad-CAM

```
def grad_cam(model, image_path, class_index):
    img = cv2.imread(image_path)
    img = cv2.resize(img, (224, 224))
    if img.shape[-1] == 1:
        img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
    img_input = np.expand_dims(img, axis=0) / 255.0

    last_conv_layer = model.get_layer("last_conv")
    grad_model = Model([model.inputs], [last_conv_layer.output,
    model.output])

    with tf.GradientTape() as tape:
```

```

        conv_outputs, predictions = grad_model(img_input)
        loss = predictions[:, class_index]

    grads = tape.gradient(loss, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

    heatmap = conv_outputs[0] @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = np.maximum(heatmap, 0)
    heatmap /= tf.math.reduce_max(heatmap)

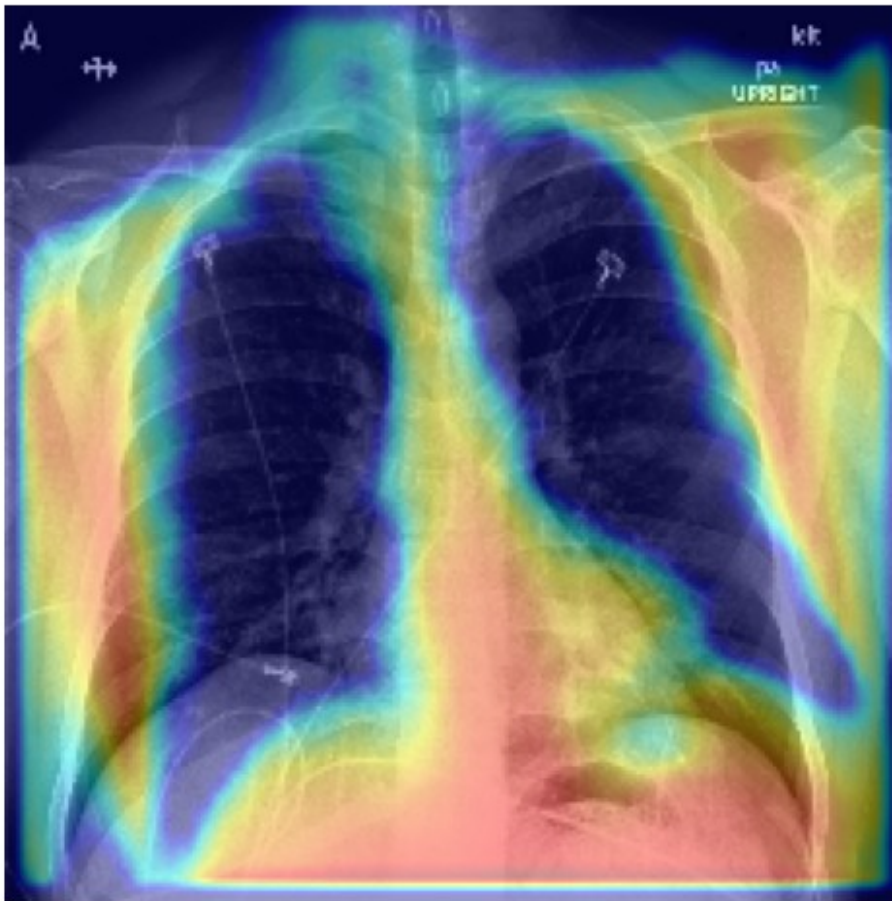
    heatmap = cv2.resize(heatmap.numpy(), (img.shape[1],
img.shape[0]))
    heatmap = np.uint8(255 * heatmap)
    heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)

    superimposed_img = cv2.addWeighted(img, 0.6, heatmap, 0.4, 0)
    plt.figure(figsize=(6, 6))
    plt.imshow(cv2.cvtColor(superimposed_img, cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.title("Grad-CAM")
    plt.show()

grad_cam(model, "lung_xray_split/train/covid/088.jpeg", class_index=0)

```

Grad-CAM



```
grad_cam(model, "/content/lung_xray_split/train/covid/08.jpeg",  
class_index=0)
```

Grad-CAM



```
grad_cam(model, "/content/lung_xray_split/train/covid/c21.jpeg",  
class_index=0)
```

Grad-CAM

